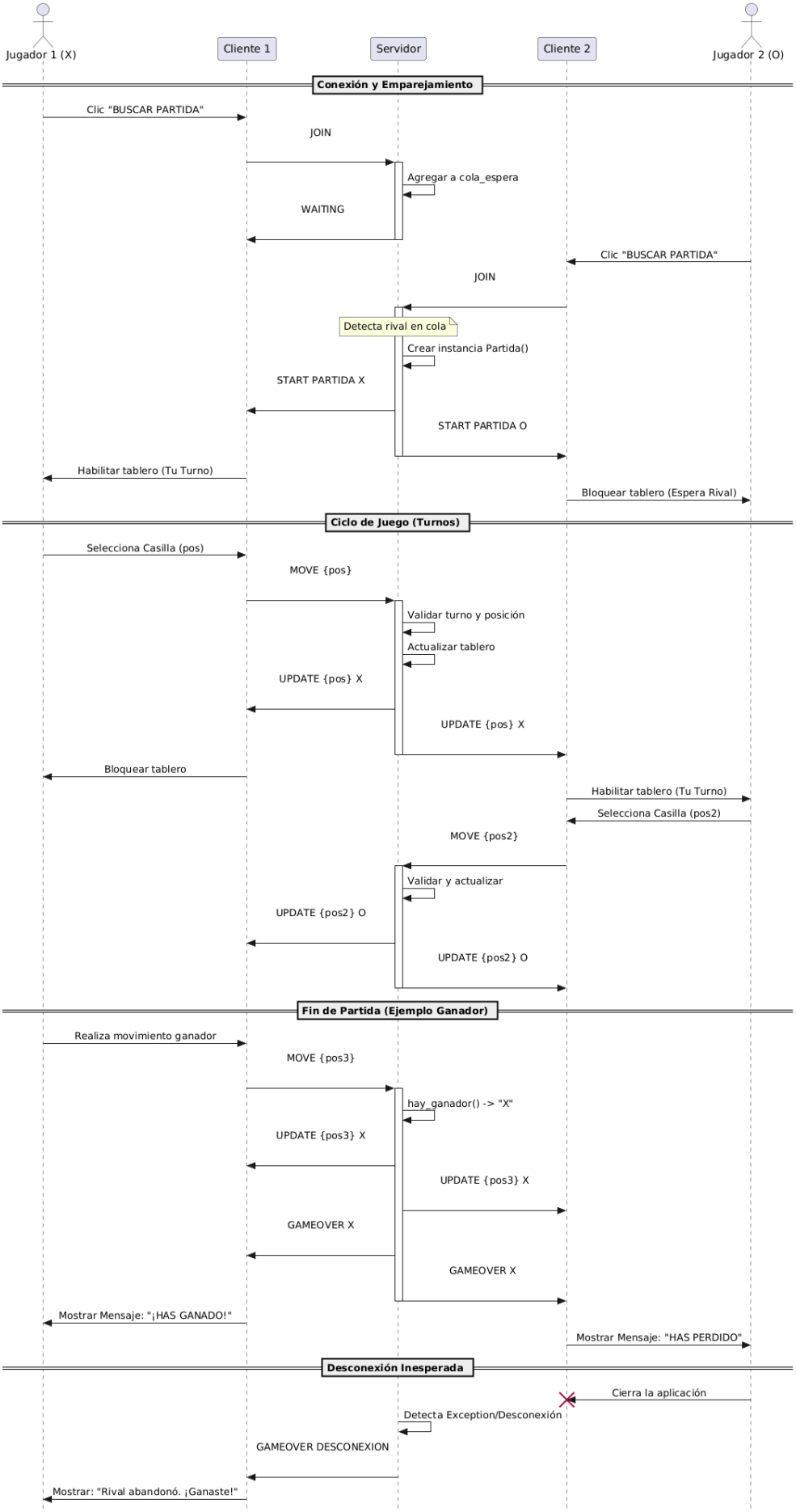
ESPECIFICACIÓN FORMAL DEL PROTOCOLO

1. Introducción El protocolo es de nivel de aplicación orientado a la conexión, diseñado para la gestión de partidas de Tres en Raya en red. Utiliza TCP como protocolo de transporte en el puerto por defecto 55555 para garantizar la entrega ordenada y fiable de los movimientos y estados del juego, evitando la pérdida de paquetes que interrumpiría la lógica del tablero.

2. Modelo de Comunicación El protocolo sigue un modelo Cliente-Servidor. El servidor actúa como una entidad de control de estado y arbitraje, mientras que los clientes funcionan como interfaces de entrada/salida para los usuarios.

- **Codificación:** UTF-8 para asegurar la compatibilidad de caracteres.
- **Delimitador de mensaje:** Secuencia de escape \r\n (CRLF) al final de cada instrucción.
- **Modo de flujo:** Bidireccional (Full-Duplex), permitiendo el envío y recepción simultánea.

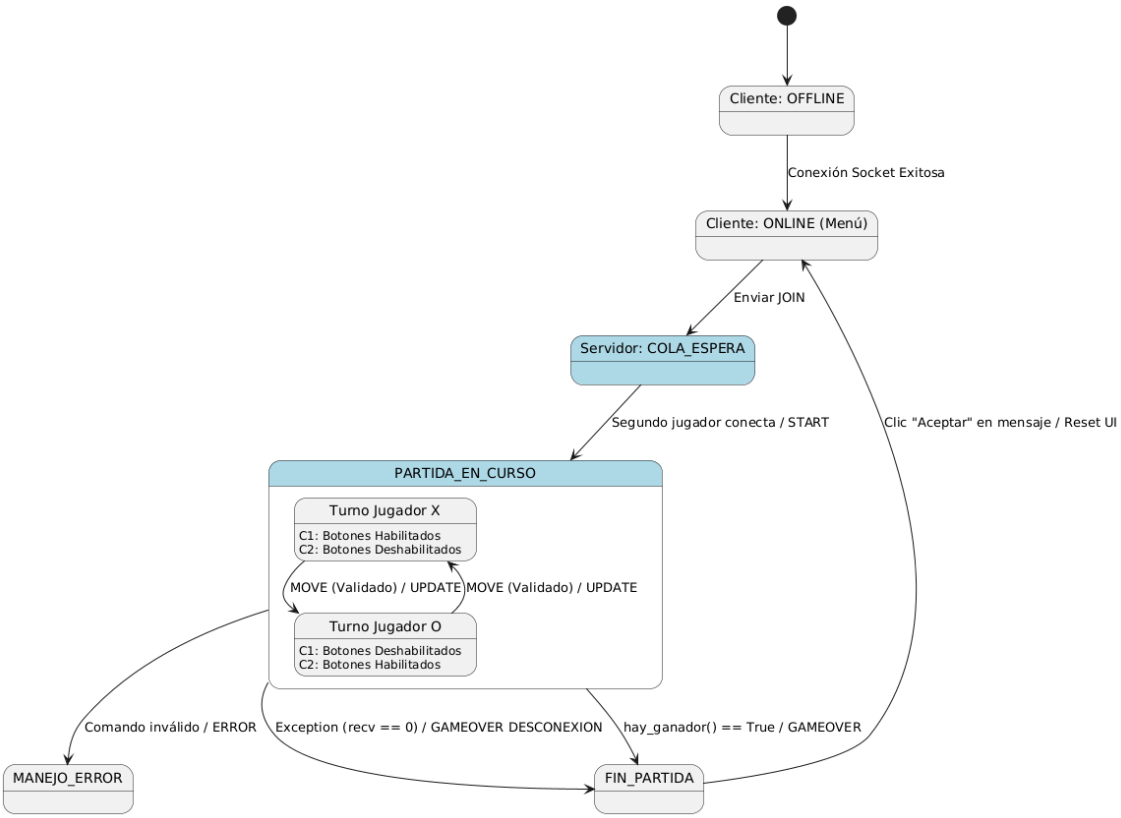
3. Diagrama de Secuencia y Flujo de Mensajes A continuación, se detalla la interacción temporal entre las entidades (Cliente A, Servidor y Cliente B) desde la fase de negociación hasta la finalización de la sesión:


Jugador 1 (X)
Cliente 1
Servidor
Cliente 2
Jugador 2 (O)
Conexión y Emparejamiento
Clic "BUSCAR PARTIDA"
JOIN
Agregar a cola_espera
WAITING
Clic "BUSCAR PARTIDA"
JOIN
Detecta rival en cola
Crear instancia Partida()
START PARTIDA X
START PARTIDA O
Habilitar tablero (Tu Turno)
Bloquear tablero (Espera Rival)
Ciclo de Juego (Turnos)
Selecciona Casilla (pos)
MOVE {pos}
Validar turno y posición
Actualizar tablero
UPDATE {pos} X
UPDATE {pos} X
Bloquear tablero
Habilitar tablero (Tu Turno)
Selecciona Casilla (pos2)
MOVE {pos2}
Validar y actualizar
UPDATE {pos2} O
UPDATE {pos2} O
Fin de Partida (Ejemplo Ganador)
Realiza movimiento ganador
MOVE {pos3}
hay_ganador() -> "X"
UPDATE {pos3} X
UPDATE {pos3} X
GAMEOVER X
GAMEOVER X
Mostrar Mensaje: "¡HAS GANADO!"
Mostrar Mensaje: "HAS PERDIDO"
Desconexión Inesperada
Cierra la aplicación
Detecta Exception/Desconexión
GAMEOVER DESCONEXION
Mostrar: "Rival abandonó. ¡Ganaste!"

4. Gramática y Sintaxis de los Comandos Los mensajes se componen de una Palabra Clave (Keyword) seguida opcionalmente de parámetros separados por espacios.

- **Comandos del Cliente (C -> S):**
  - **JOIN:** Solicita la inclusión en el sistema de emparejamiento.
  - **MOVE <pos>:** Indica un intento de movimiento. <pos> es un entero entre 0 y 8 que representa la casilla del tablero.
- **Mensajes del Servidor (S -> C):**
  - **WAITING:** Confirma que el cliente está en cola de espera.
  - **START PARTIDA <SYM>:** Notifica el inicio del juego y asigna el símbolo (X u O).
  - **UPDATE <pos> <SYM>:** Actualización de estado del tablero para ambos clientes tras un movimiento validado.
  - **GAMEOVER <MOTIVO>:** Finalización de la partida (X, O, EMPATE o DESCONEXION).
  - **ERROR <DESCRIPCION>:** Mensaje de control ante peticiones inválidas.

5. Máquina de Estados Finitos El protocolo se rige por un conjunto de estados lógicos que determinan qué comandos son válidos en cada momento. El servidor mantiene una instancia de estado por cada hilo de cliente y una instancia de partida para cada pareja de jugadores.



6. Reglas de Control y Sincronización

6.1. Validación de Turnos El servidor es el único responsable de la validación cronológica. Si un cliente envía un comando MOVE fuera de su turno, el servidor ignorará la actualización del tablero y responderá con un mensaje de ERROR. Aunque el cliente bloquea localmente su interfaz para mejorar la experiencia de usuario, la decisión final de estado reside exclusivamente en el servidor.

6.2. Gestión de Desconexiones El protocolo monitoriza el estado de la conexión TCP. Si el servidor detecta un cierre de socket (evento FIN o RST), procede a identificar la partida activa del cliente desconectado, notificar al oponente mediante el mensaje GAMEOVER DESCONEXION y liberar los recursos (Locks y punteros) asociados a dicha sesión para evitar fugas de memoria.

6.3. Concurrencia y Bloqueos Para garantizar la integridad de los datos en el acceso a la cola de emparejamiento y el diccionario de partidas activas, se implementa un mecanismo de Exclusión Mutua (Mutex) mediante la librería threading.Lock. Esto evita que dos hilos de cliente intenten modificar la lista de espera simultáneamente, previniendo condiciones de carrera.